COMP 584 — Homework 2

Dev Sanghvi
NetID: ds221

March 2, 2026

Problem 1 (25 pts): RNN Language Model

Given

$$\begin{aligned} e_{t-1} &= Ex_{t-1}, \\ a_t &= W_h h_{t-1} + W_e e_{t-1} + b_1, \\ h_t &= \tanh(a_t), \\ z_t &= Uh_t + b_2, \\ \hat{y}_t &= \mathrm{softmax}(z_t), \end{aligned}$$

with one-hot target $y_t = x_t$, per-step loss

$$L_t = -\sum_{j=1}^{V} y_{t,j} \log \hat{y}_{t,j},$$

and total loss

$$J = \sum_{t=1}^{T+1} L_t.$$

(a) Perplexity (10 pts)

By one-hot y_t, if the true token index at time t is k_t, then

$$L_t = -\log \hat{y}_{t,k_t} = -\log P(x_t \mid x_{0:t-1}).$$

Hence

$$J = -\sum_{t=1}^{T+1} \log P(x_t \mid x_{0:t-1}).$$

Perplexity is

$$\mathrm{PPL} = \exp\left(-\frac{1}{T+1}\sum_{t=1}^{T+1} \log P(x_t \mid x_{0:t-1})\right) = \exp\left(\frac{J}{T+1}\right).$$

Therefore,

$$\boxed{\mathrm{PPL} = \exp\left(\frac{J}{T+1}\right)}.$$

If the model is uniform at every step, $\hat{y}_{t,j} = 1/V$ for all t, j, then

$$P(x_t \mid x_{0:t-1}) = \frac{1}{V}, \quad L_t = -\log \frac{1}{V} = \log V.$$

So

$$J = (T+1)\log V, \quad \text{PPL} = \exp\left(\frac{(T+1)\log V}{T+1}\right) = \exp(\log V) = V.$$

Thus,

$$\boxed{\text{PPL}_{\text{uniform}} = V}.$$

(b) Backpropagation Through Time: $\partial J/\partial W_h$ (15 pts)

Define

$$\delta_t = \frac{\partial J}{\partial a_t} \in \mathbb{R}^H,$$

and let

$$g_t = \frac{\partial J}{\partial h_t}.$$

Using the provided identity:

$$\frac{\partial L_t}{\partial z_t} = \hat{y}_t - y_t.$$

Since $z_t = Uh_t + b_2$, contribution from L_t to h_t is

$$\frac{\partial L_t}{\partial h_t} = U^\top(\hat{y}_t - y_t).$$

Also h_t affects future loss through $a_{t+1} = W_h h_t + \cdots$, giving

$$\frac{\partial J}{\partial h_t} = U^\top(\hat{y}_t - y_t) + W_h^\top \delta_{t+1}.$$

Because $h_t = \tanh(a_t)$,

$$\delta_t = \frac{\partial J}{\partial h_t} \odot \left(1 - \tanh^2(a_t)\right) = \left[U^\top(\hat{y}_t - y_t) + W_h^\top \delta_{t+1}\right] \odot (1 - h_t \odot h_t).$$

With terminal condition $\delta_{T+2} = 0$, the recurrence is

$$\boxed{\delta_t = \left[U^\top(\hat{y}_t - y_t) + W_h^\top \delta_{t+1}\right] \odot (1 - h_t \odot h_t), \quad t = T+1, \ldots, 1.}$$

For W_h, since $a_t = W_h h_{t-1} + \cdots$,

$$\frac{\partial a_t}{\partial W_h} = h_{t-1}^\top,$$

so

$$\boxed{\frac{\partial J}{\partial W_h} = \sum_{t=1}^{T+1} \delta_t h_{t-1}^\top, \qquad (h_0 = 0).}$$

Problem 2 (30 pts): Understanding and Differentiating the LSTM Cell

Let

$$s_t = \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix} \in \mathbb{R}^{H+d}.$$

Assume gate parameters

$$W_f, W_i, W_o, W_g \in \mathbb{R}^{H\times(H+d)}, \quad b_f, b_i, b_o, b_g \in \mathbb{R}^H.$$

(a) Forward Pass (10 pts)

Define pre-activations

$$\begin{aligned} a_t^{(f)} &= W_f s_t + b_f, \\ a_t^{(i)} &= W_i s_t + b_i, \\ a_t^{(o)} &= W_o s_t + b_o, \\ a_t^{(g)} &= W_g s_t + b_g. \end{aligned}$$

Gate/node activations

$$\begin{aligned} f_t &= \sigma\left(a_t^{(f)}\right), \\ i_t &= \sigma\left(a_t^{(i)}\right), \\ o_t &= \sigma\left(a_t^{(o)}\right), \\ \tilde{c}_t &= \tanh\left(a_t^{(g)}\right). \end{aligned}$$

Cell and hidden updates

$$\begin{aligned} c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \\ h_t &= o_t \odot \tanh(c_t). \end{aligned}$$

Operation types:

- Affine transformations: $a_t^{(f)}, a_t^{(i)}, a_t^{(o)}, a_t^{(g)}$.
- Nonlinearities: $\sigma(\cdot)$ for gates, $\tanh(\cdot)$ for candidate and output squashing.
- Elementwise operations: $\odot$ products and vector sum in c_t.

Dimensions:

- $x_t \in \mathbb{R}^d$, $h_{t-1}, c_{t-1}, h_t, c_t \in \mathbb{R}^H$.
- $s_t \in \mathbb{R}^{H+d}$.
- $a_t^{(\cdot)}, f_t, i_t, o_t, \tilde{c}_t \in \mathbb{R}^H$.

(b) Backward Pass: Cell-State Gradient Flow (10 pts)

Define

$$\delta_t^h = \frac{\partial J}{\partial h_t}, \qquad \delta_t^c = \frac{\partial J}{\partial c_t}.$$

From

$$h_t = o_t \odot \tanh(c_t),$$

the local contribution is

$$\delta_t^c\big|_{\text{through } h_t} = \delta_t^h \odot o_t \odot \left(1 - \tanh^2(c_t)\right).$$

From

$$c_{t+1} = f_{t+1} \odot c_t + i_{t+1} \odot \tilde{c}_{t+1},$$

future contribution is

$$\delta_t^c|_{\text{through } c_{t+1}} = \delta_{t+1}^c \odot f_{t+1}.$$

Therefore,

$$\boxed{\delta_t^c = \delta_t^h \odot o_t \odot \big(1 - \tanh^2(c_t)\big) + \delta_{t+1}^c \odot f_{t+1}.}$$

This is the required recursive dependence across time (with $\delta_{T+1}^c = 0$ at sequence end).

The long-term propagation term is the additive memory path multiplied by forget gates:

$$\delta_t^c \supset \delta_T^c \odot \prod_{k=t+1}^{T} f_k,$$

which can stay stable when $f_k \approx 1$.

(c) Gradient w.r.t. Forget-Gate Parameters (5 pts)

For forget gate:

$$a_t^{(f)} = W_f s_t + b_f, \qquad f_t = \sigma(a_t^{(f)}).$$

Since $c_t = f_t \odot c_{t-1} + \cdots$,

$$\frac{\partial J}{\partial f_t} = \delta_t^c \odot c_{t-1}.$$

So

$$\delta_t^{a_f} := \frac{\partial J}{\partial a_t^{(f)}} = (\delta_t^c \odot c_{t-1}) \odot f_t \odot (1 - f_t).$$

Then

$$\boxed{\frac{\partial J}{\partial W_f} = \sum_{t=1}^{T} \delta_t^{a_f} s_t^\top.}$$

Matrix form (stacking columns):

$$\Delta_f = [\delta_1^{a_f}, \ldots, \delta_T^{a_f}] \in \mathbb{R}^{H \times T}, \quad S = [s_1, \ldots, s_T] \in \mathbb{R}^{(H+d) \times T}, \quad \frac{\partial J}{\partial W_f} = \Delta_f S^\top.$$

(d) Stability Analysis (5 pts)

From

$$c_k = f_k \odot c_{k-1} + \cdots,$$

the Jacobian between cell states is

$$\frac{\partial c_T}{\partial c_t} = \prod_{k=t+1}^{T} \operatorname{diag}(f_k).$$

If $f_k \approx 1$ for all k, then

$$\boxed{\frac{\partial c_T}{\partial c_t} \approx I.}$$

For a vanilla RNN,

$$h_k = \phi(W_h h_{k-1} + \cdots), \quad \frac{\partial h_T}{\partial h_t} = \prod_{k=t+1}^{T} D_k W_h,$$

where $D_k = \mathrm{diag}(\phi'(\cdot))$. This repeated matrix product often shrinks (or explodes) with time due spectral effects and saturating nonlinearities.

Hence LSTMs mitigate vanishing gradients by providing an additive memory path with near-identity Jacobian when forget gates stay near 1.

Problem 3 (45 pts): Implementing an LSTM from Scratch

All implementation details and executable code are provided in:

`HW2.ipynb`

(a) Single LSTM Cell (5 pts)

Implemented class:

`LSTMCellScratch(nn.Module)`

with forward signature

`forward(x_t, h_prev, c_prev) -> (h_t, c_t).`

Implementation properties:

- Uses affine transforms over concatenated input $[h_{t-1}; x_t]$.
- Explicit forget, input, output, and candidate gates.
- Cell update and hidden update are implemented as separate elementwise operations.

(b) Sequence Model (10 pts)

Implemented language model:

- **Embedding layer**: maps token ids to dense vectors.
- **Custom LSTM cell**: unrolled over sequence length.
- **Linear output layer**: maps each h_t to vocabulary logits.

The forward pass returns logits for all time steps with shape

`[batch_size, seq_len, vocab_size].`

(c) Synthetic Copy Task Results (20 pts)

Task setup: input sequence equals target sequence,

$$(x_1, \ldots, x_T) \mapsto (x_1, \ldots, x_T).$$

Training configuration used in the executed notebook:

Hyperparameter	**Value**
Vocabulary size	30
Embedding dimension	32
Hidden size	64
Sequence length T	30
Batch size	128
Optimizer	Adam
Learning rate	3×10^{-3}
Epochs	20
Steps per epoch	60

Token sampling protocol: tokens are sampled uniformly over all vocabulary IDs from 0 to $V-1$ (no token ID is excluded).

Measured outcomes (from executed run):

- Final validation loss: 0.00029591
- Final validation perplexity: 1.00029595
- Final validation token accuracy: 1.0000
- Cell-state gradient norm range: min $= 6.31 \times 10^{-8}$, max $= 6.867 \times 10^{-7}$

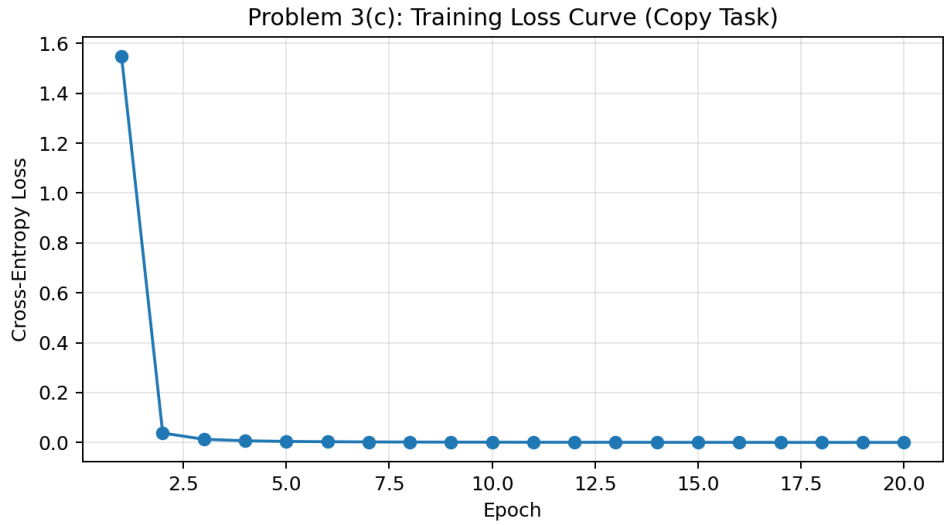

Figure 1: Training loss curve for the scratch LSTM on the copy task.

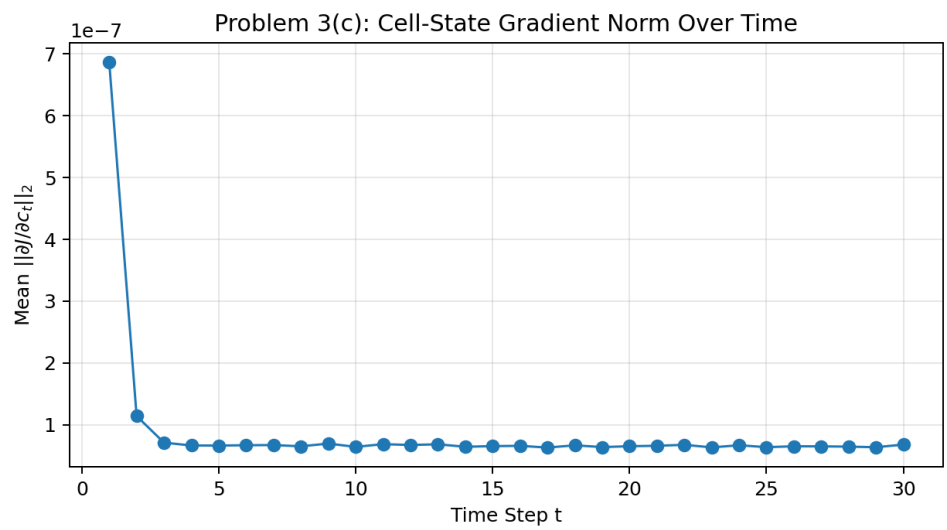

Figure 2: Cell-state gradient norm over time steps.

(d) Q&A (10 pts)

Vanilla RNNs face several structural limitations in NLP. First, sequence processing is strictly recurrent, so computation is sequential across time and cannot be fully parallelized during training; this limits throughput on long texts. Second, gradients pass through many repeated Jacobian multiplications, which makes optimization unstable due vanishing or exploding gradients, especially

for long contexts. Third, hidden-state compression into a fixed-size vector bottleneck can lose fine-grained token interactions and long-range dependencies. Fourth, recurrence has a long effective path length between distant tokens, so learning dependencies across far-apart words is difficult.

Transformers address these issues with self-attention. Every token can directly attend to every other token in the same layer, so long-distance interactions have short gradient paths. Attention and feed-forward blocks are highly parallelizable across token positions, greatly improving hardware utilization and training speed. Multi-head attention captures multiple relation types simultaneously, and deep stacks with residual connections plus normalization improve optimization stability. With positional encodings and scaling, Transformers model both local and global context effectively, which is why they outperform RNNs on modern NLP tasks.

Submission files prepared:

- `HW2.pdf` (this report)
- `HW2.ipynb` (full coding implementation and outputs)
- `NLP_HW2_ds221.zip` (submission bundle)